



Laboratorio 03 — Legacy System Update

David F. Alonso¹

1. Código: 160005001.

Programa de Ingeniería de Sistemas.

Resumen—El presente proyecto aborda la migración y refactorización de un módulo de gestión bancaria legacy programado originalmente en C++ hacia una arquitectura robusta orientada a objetos en Java 21. El objetivo principal consistió en erradicar vulnerabilidades de gestión de memoria (como memory leaks y punteros colgados) inherentes al sistema anterior y estructurar un dominio financiero escalable que contemple cuentas de ahorro y cuentas corrientes mediante abstracciones, encapsulamiento, polimorfismo y administración segura de recursos. La práctica se apoyó en un proceso de auditoría asistido de manera socrática mediante Inteligencia Artificial (Gemini Gema).

Palabras clave—C++, refactorización, legacy, memory leaks, encapsulamiento, polimorfismo.

I. INTRODUCCIÓN

La práctica de laboratorio permitió realizar un primer acercamiento al uso de Git y GitHub como herramientas fundamentales para el control de versiones y la colaboración en proyectos de desarrollo de software. A través de la creación y gestión de un repositorio, la realización de commits, el trabajo con ramas y la utilización de Pull Requests, se desarrolló un flujo básico de trabajo empleado en proyectos reales de ingeniería de software.

Durante la práctica se siguió el instructivo oficial de GitHub Start your journey, aplicando los conceptos de control de versiones directamente sobre un repositorio público. Esto permitió observar cómo Git registra los cambios realizados en un proyecto y cómo GitHub facilita su almacenamiento y colaboración.

II. AUDITORÍA TÉCNICA DEL CÓDIGO LEGACY EN C++ (FASE I)

A. Identificación de Fugas de Memoria y Punteros Colgados

En la implementación inicial del código C++ (SistemaBancarioLegacy.cpp), se identificaron deficiencias críticas de gestión de memoria dinámica:

C/C++

```
CuentaLegacy* crearCuenta(const char* nombre, double saldoInicial, int tipo) {
    CuentaLegacy* c = new CuentaLegacy();
    c->titular = new char[50];
    ...
}
```

1) *Fuga de Memoria en crearCuenta*: Se reserva memoria dinámica en el Heap mediante `new char[50]`. Al no existir una función de destrucción que ejecute `delete[] c->titular`, dicha memoria queda permanentemente asignada hasta la finalización del proceso.

2) *Ausencia de Desasignación (`delete` / `delete[]`)*: El programa finaliza sus ejecuciones sin liberar la memoria asignada dinámicamente. El puntero titular queda huérfano, provocando fugas de memoria persistentes (memory leaks) que degradarían progresivamente la disponibilidad de RAM en un entorno bancario de alta disponibilidad.

3) *Deficiencia de Encapsulamiento*: La estructura struct CuentaLegacy expone directamente sus campos públicos (saldo, limiteSobregiro, etc.), permitiendo mutaciones de estado no autorizadas desde cualquier parte del programa sin pasar por validaciones de negocio.

B. Comparativa Arquitectónica: C++ vs. Java

Criterio	Sistema Legacy (C++)	Refactorización Modernizada (Java 21)
Paradigma	Procedimental estructurado sobre struct.	Orientado a Objetos puro (Clases abstractas e interfaces).
Gestión de Memoria	Manual mediante el Heap (new/delete). Riesgo alto de memory leaks.	Automática administrada por el Garbage Collector (GC) por rastreo de alcanzabilidad.
Protección de Datos	Atributos públicos manipulables directamente.	Atributos private/final con acceso controlado mediante getters y setters protegidos.
Polimorfismo	Control de tipo explícito e inseguro mediante switch/if sobre flags (tipoCuenta).	Despacho dinámico puro con sobreescritura de métodos abstractos (invokevirtual).

III. REDISEÑO ORIENTADO A OBJETOS EN JAVA 21 (FASE 2)

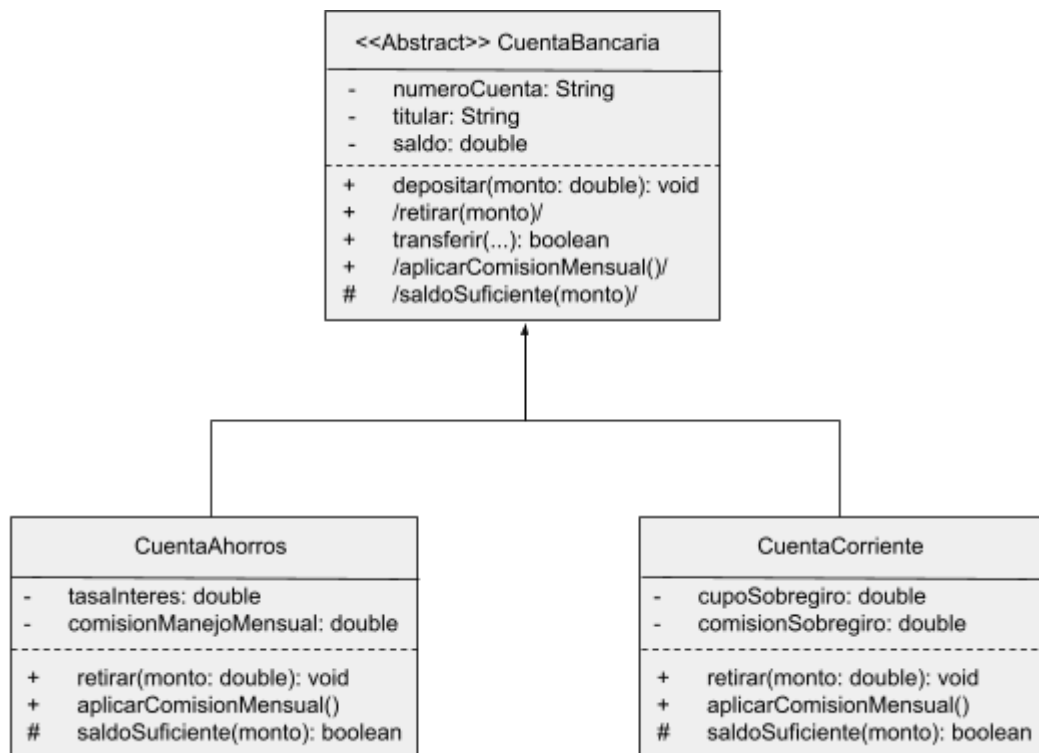
El diseño refactorizado elimina las estructuras de control condicionales para determinar reglas financieras, migrando hacia una jerarquía polimórfica:

A. Estructura de la Jerarquía de Clases

1) *CuentaBancaria (Clase Abstracta Base)*: Encapsula los campos comunes titular, numeroCuenta y saldo. Declara titular y numeroCuenta como inmutables (final) y restringe la modificación directa del saldo mediante un setter con visibilidad protegida (protected).

2) *CuentaAhorros (Subclase Concreta)*: Define una tasa de interés mensual y una comisión fija de manejo. Sobreescrive saldoSuficiente(monto) para prohibir estrictamente los sobregiros (el saldo no puede ser inferior a 0.0).

3) *CuentaCorriente (Subclase Concreta)*: Modela el crédito de liquidez mediante el atributo cupoSobregiro y la constante comisionSobregiro. Permite que el saldo tome valores negativos hasta el límite autorizado (-(cupoSobregiro)).



IV. GESTIÓN DE RECURSOS BANCARIOS Y AUDITORÍA (FASE 3)

A. Sustitución de RAII por AutoCloseable

En C++, el patrón RAII (Resource Acquisition Is Initialization) garantiza la liberación de recursos (archivos o descriptores) mediante el destructor al salir del ámbito de la variable. En Java, la gestión automática del Garbage Collector no garantiza la liberación inmediata de recursos del sistema operativo.

Para resolver esta problemática, la clase RegistroAuditoriaBancaria implementa la interfaz `java.lang.AutoCloseable`.

B. Ventajas de la Sintaxis *try-with-resources*

1) *Cierre Garantizado de Recursos*: Al instanciar RegistroAuditoriaBancaria dentro del encabezado `try (...)`, la máquina virtual de Java (JVM) invoca automáticamente el método `.close()` al salir del bloque de código, sin importar si la ejecución fue exitosa o interrumpió por una excepción no controlada.

2) *Eliminación del Bloque *finally* Extenso*: Define una tasa de interés mensual y una comisión fija de manejo. Sobreescribe `saldoSuficiente(monto)` para prohibir estrictamente los sobregiros (el saldo no puede ser inferior a 0.0).

V. CONCLUSIONES

- La migración del código C++ procedimental hacia una arquitectura POO en Java 21 resolvió de forma definitiva los problemas de fugas de memoria y fragilidad de encapsulamiento.
- La aplicación de polimorfismo permitió centralizar las operaciones comunes de transferencia y depósito en la clase base CuentaBancaria, delegando las variaciones de negocio de sobregiro y comisiones a las subclases especializadas.
- La implementación de AutoCloseable y la estructura *try-with-resources* proporcionaron una gestión determinista y segura de archivos de auditoría de transacciones, equivalente a las garantías del patrón RAII en C++.

VII. REPOSITORIO

Repositorio del laboratorio: *laboratorio_03_Legacy-System-Update*

Enlace: https://github.com/Labs-Software-I/laboratorio_03_Legacy-System-Update.git